
Laborbuch

SafetySpot

Stand	29.06.2026
Kunde	Bildungsministerium NRW
Projektbezeichnung	SafetySpot
Autor / Kürzel	D0rag3on (Noel Neuhoff) IIRu5hII (Milian) 0qIn
Art des Dokuments	Persönliches Laborbuch

1 Grundlage und Projektkontext

Dieses Dokument vereint die persönlichen Laborbücher aller drei Projektteilnehmer des Projekts *SafetySpot* in einer einzigen, chronologisch sortierten Tabelle. Einträge werden nicht nach Autor getrennt, sondern rein nach Datum einsortiert. Der Autor ist in jeder Zeile explizit ausgewiesen.

Die Einträge von IIRu5hII ab Januar 2026 wurden sachlich aus Projektdefinition, Mockups und Quellcode rekonstruiert, da frühe Arbeitsschritte ohne Commits erfolgten. Alle anderen Einträge basieren direkt auf Git-Commits und GitHub-Pull-Requests im Repository *safety-spot/SafetySpot*.

2 Chronologische Laborbuch-Einträge (alle Autoren)

Datum	Autor	Klassifikation	Beschreibung / Ergebnis
2026-01-12	IIRu5hII	Recherche	Eintrag 01: Projektdefinition gesichtet und Ausgangslage festgehalten. Gefahrenaufklärung an Schulen erfolgt häufig nur mündlich und wenig interaktiv. Daraus wurde abgeleitet, dass die App niedrigschwellig, spielerisch und direkt verständlich sein muss. <i>Ergebnis:</i> Projektdefinition als fachliche Grundlage; Fokus auf Schüler, Lehrer und Schulen.
2026-01-12	IIRu5hII	Ziel	Eintrag 02: Zielbild für die mobile App formuliert: Schüler sollen Gefahrensituationen in einer sicheren Umgebung bewerten, daraus lernen und durch Punkte/Rankings motiviert bleiben. <i>Ergebnis:</i> Zielbild für Frontend, Szenarien und Gamification übernommen.
2026-01-12	IIRu5hII	Annahmen/Hypothese	Eintrag 03: Annahme: Ein spielerischer Ablauf mit kurzen Aufgaben und sofortigem Feedback ist für Schüler motivierender als reine Sicherheitsbelehrungen. <i>Ergebnis:</i> Beeinflusst Spielscreen mit zwei großen Antwortbuttons und Feedback.
2026-01-22	IIRu5hII	Entscheidung	Eintrag 04: Entwicklungsumgebung festgelegt: Android Studio. Alternativen (IntelliJ IDEA, Qt Creator, Cordova) als weniger passend bewertet. <i>Ergebnis:</i> Android Studio als gemeinsame Arbeitsumgebung eingeplant.
2026-01-23	IIRu5hII	Entscheidung	Eintrag 05: Programmiersprache für Android-App: Kotlin. Gründe: moderne Android-Entwicklung, Null-Safety, weniger Boilerplate, gute Integration in Android Studio. <i>Ergebnis:</i> Kotlin als Basis für die mobile App.
2026-01-23	IIRu5hII	Recherche	Eintrag 06: Informationsmodell ausgewertet. Relevante Datenblöcke: Organisation/Rollen, Lerninhalte, Fortschritt/Gamification und Admin/Lizenz. <i>Ergebnis:</i> Datenmodell als Grundlage für spätere DTOs, Mockdaten und UI-Strukturen.

Datum	Autor	Klassifikation	Beschreibung / Ergebnis
2026-01-23	IIRu5hII	Entscheidung	Eintrag 07: Grundsatzentscheidung für Server-als-Wahrheit übernommen. Backend speichert alles; App hält nur Token, Cache und ggf. ausstehende Versuche. <i>Ergebnis:</i> Frühe Frontend-Version trotzdem mit Mockdaten aufgebaut.
2026-01-24	IIRu5hII	Ziel	Eintrag 08: Projektvision analysiert. Erstes Release: lauffähige Android-App mit interaktiven Szenarien, Punkten/Ranking und einfachem Admin-Gedanken. <i>Ergebnis:</i> Scope auf wichtigste Schüler-Funktionen fokussiert.
2026-01-24	IIRu5hII	Idee	Eintrag 09: Kategorien aus Vision und Mockups abgeleitet: Chemieraum, Werkraum, Sportunterricht, Technikraum, Straßenverkehr. <i>Ergebnis:</i> Kategorien in sampleScenarios und Home-Screen sichtbar.
2026-01-24	IIRu5hII	Bewertung	Eintrag 10: Risiken gesichtet: Android-only, Leaderboard-Sozialdruck, zu komplexe eigene Szenarien. <i>Ergebnis:</i> UI einfach gehalten; Leaderboard motivierend statt bloßstellend.
26.04.2026	D0rag3on	Versuchsaufbau	Eintrag 11: Initialisierung des Server-Projekts unter dem vorläufigen Namen SafetySpotServer. Maven-basiertes Spring-Boot-Projekt inkl. mvnw, pom.xml, persistence.xml, beans.xml. Zu diesem Zeitpunkt noch kein funktionaler Code – lediglich ein lauffähiger, leerer Startpunkt als Grundlage für die Backend-Entwicklung.
26.04.2026	Oqln	Versuchsaufbau	Eintrag 12: Repository safety-spot/SafetySpot initialisiert (Commit 183ac5f). Grundlegende Repo-Struktur: .gitignore, .gitattributes, README.md; Zeilenendenrichtlinien (.gitattributes) für plattformübergreifende Zusammenarbeit normalisiert. <i>Ergebnis:</i> Gemeinsames Repository als Startpunkt für das gesamte Team.
09.06.2026	D0rag3on	Versuchsaufbau	Eintrag 13: Migration des Backend-Projekts von Maven auf Gradle sowie Umbenennung von SafetySpotServer auf ssbackend. Das gesamte Projekt verwendet Gradle als einheitliches Build-System; Maven war ein inkonsistenter Fremdkörper. 316 Zeilen (mvnw, pom.xml, IDE-Konfiguration) entfernt. Neues Modul mit Spring-Boot-Gradle-Plugin und SsbackendApplication.java aufgebaut. <i>Anmerkung:</i> Der zugehörige erste Commit hätte früher erfolgen sollen.
09.06.2026	Oqln	Versuchsaufbau	Eintrag 14: Versionsdokumentation eingepflegt – v0-Dokumente (Projektvision, Informationsmodell, Entscheidungen, Risiken, Klassendiagramm docs/v0) und v1-Mockups (docs/v1) dem Repository hinzugefügt (Commits cf734b4, cf9eabf). <i>Ergebnis:</i> Fachliche Grundlage und UX-Mockups im Repo versioniert; alle Teammitglieder haben Zugriff ohne separaten Dateiumlauf.

Datum	Autor	Klassifikation	Beschreibung / Ergebnis
09.06.2026	OqIn	Versuchsaufbau	Eintrag 15: sscommon-Kotlin-Modul initiiert sowie Copilot-Instruktionen und Projekt-Spezifikation angelegt (PR #26, 1a90de6). Inhalt: Copilot-Instruktionsdatei <code>.github/copilot-instructions.md</code>, sscommon/-Gradle-Modul (geteilte Bibliothek), <code>spec/plan.md</code> (Backend-Implementierungsplan mit Paketen, DTOs, Security-Konzept), ISSUE-001 bis ISSUE-011 (Backend-Aufgaben). Außerdem: Dev-Umgebung als Nix-Flake vorbereitet. <i>Ergebnis:</i> Strukturierte Aufgabenplanung und Copilot-Wissen als Grundlage für alle weiteren Backend-Entwicklungsschritte.
09.06.2026	OqIn	Versuchsaufbau	Eintrag 16: ssmobile-Android-Projekt initialisiert (PR #27, Commit 6caaf64). Gradle-Build-Konfiguration, <code>AndroidManifest.xml</code>, Basis-Theme (Material3), Basis-Navigation und <code>MainActivity.kt</code>. Struktur angelegt für spätere UI-Screens. <i>Ergebnis:</i> Lauffähiges Android-Projekt-Skelett; IIRu5hII kann sofort auf der gleichen Basis aufbauen.
2026-06-10	IIRu5hII	Ziel	Eintrag 17: Start der praktischen Frontend-Arbeit ohne Backend-Integration. Ziel: klickbarer Android-Prototyp, der Mockups nachbildet und den Kernablauf von Login bis Szenario-Spiel zeigt. <i>Ergebnis:</i> Rekonstruiert aus Codezustand und Autorenaussage.
2026-06-10	IIRu5hII	Aufgaben	Eintrag 18: Android-Projekt geöffnet, Compose-Grundstruktur geprüft, <code>MainActivity/SafetySpotApp</code> vorbereitet. Erste Orientierung an Package-Struktur <code>ui/auth</code>, <code>ui/home</code>, <code>ui/scenarios</code>, <code>ui/ranking</code>, <code>ui/profile</code>. <i>Ergebnis:</i> Projekt kann als mobile Compose-App weiterentwickelt werden.
2026-06-10	IIRu5hII	Ergebnis	Eintrag 19: Grundlegendes Design-System aufgebaut: Markenfarben, App-Hintergrund, Kartenfarben, Kategorie-Akzente und Schwierigkeitsfarben. <i>Ergebnis:</i> <code>Color.kt</code> enthält <code>BrandGreen</code>, <code>BrandBlue</code>, <code>PointsYellow</code>, <code>ChemieBlueTint</code>, <code>WerkraumOrange</code>, <code>SportGreen</code>, <code>TechnikPurple</code>, <code>TrafficRed</code>.
2026-06-11	IIRu5hII	Aufgaben	Eintrag 20: Navigationskonzept umgesetzt: Auth als Startscreen, Hauptnavigation mit Home/Szenarien/Ranking/Profil sowie Detailrouten für Szenario-Detail, Gameplay und Profil-Unterseiten. <i>Ergebnis:</i> <code>SafetySpotApp</code> enthält <code>NavHost</code> und <code>BottomBar</code>-Logik.
2026-06-11	IIRu5hII	Ergebnis	Eintrag 21: Bottom Navigation aufgebaut. Leiste wird nur auf Hauptscreens angezeigt, auf Detail-/Spielseiten ausgeblendet. <i>Ergebnis:</i> Navigation entspricht Mockups.

Datum	Autor	Klassifikation	Beschreibung / Ergebnis
2026-06-11	IIRu5hII	Ergebnis	Eintrag 22: AuthScreen erstellt. Enthält Branding, Login-/Registrieren-/ Gastzugang-Logik als UI-Prototyp; Einstieg in die App. <i>Ergebnis:</i> Frontend ohne echte Zugangsdaten testbar.
11.06.2026	Oqln	Ergebnis	Eintrag 23: Projekt-weite Struktur-Überarbeitung (PR #28, 9ee221e). Anpassungen an Build-Skripten und Projekt-Struktur nach erstem Feedback und paralleler Backend-Entwicklung. <i>Ergebnis:</i> Konsistente Projektstruktur über alle drei Module.
12.06.2026	D0rag3on	Versuchsaufbau	Eintrag 24: Anlegen der Paket- und Klassenstruktur für globales Error-Handling. Neue Pakete: dto, exception. Erstellt als initiale Stubs: ErrorResponse (einheitliches DTO), AccessDeniedException, EntityNotFoundException, GlobalExceptionHandler. Alle Klassen bewusst leer gehalten – Struktur vor Implementierung.
2026-06-12	IIRu5hII	Ergebnis	Eintrag 25: HomeScreen aufgebaut: Begrüßung, Level, Punkte, Streak, Weiterlernen-Bereich und Kategorien. <i>Ergebnis:</i> Zentraler Einstieg für Schüler nach Login.
2026-06-12	IIRu5hII	Ergebnis	Eintrag 26: Wiederverwendbare Komponenten begonnen: MetricCard, StreakRow, SafetyProgressBar, SectionHeader, CategoryTile, ScenarioCard. <i>Ergebnis:</i> Reduziert Dopplungen; spätere Anpassungen einfacher.
2026-06-12	IIRu5hII	Idee	Eintrag 27: Für erste UI-Version bewusst Mockdaten genutzt, da Backend noch nicht vollständig angebunden. <i>Ergebnis:</i> Mockdaten werden schrittweise durch API-Daten ersetzt.
2026-06-13	IIRu5hII	Ergebnis	Eintrag 28: ScenariosScreen: Titel, Suchfeld, Filtertabs (Alle/Neu/Beliebt/Abgeschlossen) und Szenario-Kartenliste. <i>Ergebnis:</i> Nutzer kann Szenarien visuell auswählen und filtern.
2026-06-13	IIRu5hII	Ergebnis	Eintrag 29: sampleScenarios angelegt: Chemieraum, Werkraum, Sportunterricht, Straßenverkehr, Technikraum. <i>Ergebnis:</i> Entspricht Kategorien aus Projektvision und Mockups.
2026-06-13	IIRu5hII	Problem	Eintrag 30: Umlaute und Sonderzeichen können in Code/Assets zu Darstellungsproblemen führen. <i>Ergebnis:</i> Teilweise ASCII-Schreibweisen in Mockdaten verwendet.
14.06.2026	D0rag3on	Messergebnis	Eintrag 31: Vollständige Implementierung des GlobalExceptionHandler (45 Zeilen, @RestControllerAdvice). Alle definierten Ausnahmetypen werden auf HTTP-Statuscodes gemappt und als ErrorResponse-DTO zurückgegeben. Das globale Error-Handling ist damit abgeschlossen und einsatzbereit.

Datum	Autor	Klassifikation	Beschreibung / Ergebnis
2026-06-14	IIRu5hII	Ergebnis	Eintrag 32: ScenarioDetailScreen erstellt: Kategorie, Titel, Schwierigkeit, Aufgabenanzahl und Status vor dem Spielstart. <i>Ergebnis:</i> Zwischenschritt zwischen Szenarienliste und Gameplay.
2026-06-14	IIRu5hII	Ergebnis	Eintrag 33: ScenarioPlayScreen aufgebaut: Fortschrittsanzeige, Punkteanzeige, Aufgabeninhalt, Kategorie-/Mascot-Bereich und große Antwortbuttons (gefährlich / nicht gefährlich). <i>Ergebnis:</i> Kernfunktion der App im Prototyp spielbar.
2026-06-14	IIRu5hII	Annahmen/Hypothese	Eintrag 34: Hypothese: Zwei große Antwortbuttons sind für Schulkinder verständlicher als komplexes Multiple-Choice. <i>Ergebnis:</i> True/False-Modus für Version 1 priorisiert.
15.06.2026	D0rag3on	Versuchsaufbau	Eintrag 35: Anlegen des vollständigen Projektskeletts für ssbackend: 25 Java-Klassen über alle fachlichen Pakete verteilt. Pakete: attempt (8 Klassen), auth (2), enums (3), leaderboard (3), progress (2), school (3), user (4). Alle Klassen als Stubs; Ziel: Gesamtstruktur des Backends abbilden, bevor Implementierung beginnt.
2026-06-15	IIRu5hII	Ergebnis	Eintrag 36: Feedbacklogik im Spielscreen ergänzt. Nach einer Antwort: korrekt/falsch hervorgehoben, Feedbacktext angezeigt. <i>Ergebnis:</i> Direktes Feedback unterstützt Lerneffekt.
2026-06-15	IIRu5hII	Ergebnis	Eintrag 37: Lokaler Punktestand und Abschlusslogik eingeführt. Nach Abschluss eines Szenarios: Punkte addiert, Szenario als abgeschlossen markiert. <i>Ergebnis:</i> Gamification-Effekt ohne Backend-Persistenz.
2026-06-15	IIRu5hII	Bewertung	Eintrag 38: Mock-first-Ansatz bewertet: Vorteil: schneller UI-Fortschritt; Nachteil: spätere Umstellung auf echte API-Modelle. <i>Ergebnis:</i> Entscheidung bleibt sinnvoll; Prototyp früh prüfbar.
2026-06-16	IIRu5hII	Ergebnis	Eintrag 39: RankingScreen: Scope-Tabs (Klasse/Schule/Weltweit), Top-3-Darstellung, Liste weiterer Spieler mit Hervorhebung des aktuellen Nutzers. <i>Ergebnis:</i> Gamification-Ziel Punkte/Ranking sichtbar.
2026-06-16	IIRu5hII	Ergebnis	Eintrag 40: sampleLeaderboard angelegt und dynamisch mit aktuellem Nutzer kombiniert. Lokaler Punktestand kann den Rang beeinflussen. <i>Ergebnis:</i> Ranking reagiert im Prototyp auf abgeschlossene Szenarien.
2026-06-16	IIRu5hII	Risiko/Beobachtung	Eintrag 41: Leaderboard kann motivieren, aber auch Druck erzeugen. <i>Ergebnis:</i> Eigener Nutzer positiv hervorgehoben; spätere Option: Klassenranking abschaltbar.

Datum	Autor	Klassifikation	Beschreibung / Ergebnis
2026-06-17	IIRu5hII	Ergebnis	Eintrag 42: ProfileScreen erstellt: Avatar, Name, Level, XP-Balken, Punkte, Streak, Abzeichen und Menügruppen. <i>Ergebnis:</i> Profil macht Fortschritt und Gamification persönlich sichtbar.
2026-06-17	IIRu5hII	Ergebnis	Eintrag 43: Profil-Unterseiten ergänzt: Mein Fortschritt, Abzeichen und Meine Szenarien. <i>Ergebnis:</i> Nutzer kann Szenarien und Auszeichnungen nachvollziehen.
2026-06-17	IIRu5hII	Ergebnis	Eintrag 44: SettingsScreen und HelpFeedbackScreen ergänzt. <i>Ergebnis:</i> Erhöht Eindruck einer vollständigen App.
2026-06-18	IIRu5hII	Aufgaben	Eintrag 45: Mockup-Screens mit realem Compose-Layout abgeglichen. Abstände, Karten, Rundungen und Farben angepasst. <i>Ergebnis:</i> UI wirkt wie Projekt-Mockups, nicht wie Standard-Demo.
2026-06-18	IIRu5hII	Ergebnis	Eintrag 46: Szenario-/Kategoriedaten verfeinert. Chemie, Werkraum, Sport, Technik und Verkehr als Lernbereiche vorhanden. <i>Ergebnis:</i> Struktur lässt sich durch Backend-Kategorien erweitern.
2026-06-18	IIRu5hII	Idee	Eintrag 47: SafetySpot-Drache/Maskottchen als unterstützendes Element vorgesehen. Für Feedback, Motivation und kindgerechte Ansprache. <i>Ergebnis:</i> Mascot-Bereich im Spielscreen vorgesehen; echte Assets später.
18.06.2026	OqIn	Aufgaben	Eintrag 48: Mobile-Spezifikations-Issues ISSUE-012 bis ISSUE-022 erstellt (Commits 2161ee6-701aa4c). Themen: Foundation/Build-Config, Design-System, Networking/Auth-Infrastruktur, Navigation-Shell, Auth-Screen, Home-Screen, Szenarios-Browse, Szenario-Gameplay, Ranking-Screen, Profil-Screen, Offline-Cache-Sync. <i>Ergebnis:</i> Vollständige Spezifikation der mobilen App-Screens als Grundlage für IIRu5hII's Frontend-Entwicklung und eigene Clickdummy-Arbeit.
2026-06-19	IIRu5hII	Ergebnis	Eintrag 49: UI-Komponenten weiter standardisiert: einheitliche Cards, Chips, ProgressBars, Textfarben und Buttons. <i>Ergebnis:</i> Verbessert Wartbarkeit und konsistente Darstellung.
2026-06-19	IIRu5hII	Problem	Eintrag 50: Durch lokale UI-Zustände drohten inkonsistente Punkte/ abgeschlossene Szenarien beim Navigieren. <i>Ergebnis:</i> Zustand in SafetySpotApp zentralisiert: sessionPoints und completedScenarioIds.

Datum	Autor	Klassifikation	Beschreibung / Ergebnis
2026-06-19	IIRu5hII	Lösung	Eintrag 51: completedScenarioIds als mutableStateListOf, sessionPoints als Compose-State. <i>Ergebnis:</i> Lokaler Prototyp ohne Backend trotzdem konsistent.
2026-06-20	IIRu5hII	Ergebnis	Eintrag 52: Navigation zwischen Szenario-Liste, Detailansicht und Gameplay geprüft und verbessert. <i>Ergebnis:</i> Back-Navigation und Starten eines Szenarios funktionieren.
2026-06-20	IIRu5hII	Test/Beobachtung	Eintrag 53: Manueller Test des Kernflows: Auth → Home → Szenarien → Detail → Spielen → Ergebnis → Ranking/Profil. <i>Ergebnis:</i> Kompletter Schüler-Flow im Frontend testbar.
2026-06-20	IIRu5hII	Offene Punkte	Eintrag 54: Persistenz fehlt: Punktestand und Abschlussstatus gehen nach App-Neustart verloren. <i>Ergebnis:</i> Für nächste Stufe: Backend/Room/DataStore.
2026-06-21	IIRu5hII	Ergebnis	Eintrag 55: Profil- und Rankingdaten mit lokalen Spielständen verknüpft. <i>Ergebnis:</i> Gamification fühlt sich zusammenhängender an.
2026-06-21	IIRu5hII	Bewertung	Eintrag 56: Frontend-Prototyp erfüllt wichtigste UI-Anforderungen: simpel, mobil, visuell freundlich, große Bedienelemente, spielerischer Ablauf. <i>Ergebnis:</i> Basis für Demo und spätere Backend-Integration vorhanden.
2026-06-21	IIRu5hII	Aufgaben	Eintrag 57: Code bereinigt und in UI-Pakete aufgeteilt: auth, home, scenarios, scenario, ranking, profile, navigation, theme, components. <i>Ergebnis:</i> Ordnung erleichtert Teamarbeit.
21.06.2026	OqIn	Aufgaben	Eintrag 58: Frontend-Planung dokumentiert (PR #43, d01cb56). Entscheidungen zu UI-Architektur, Screen-Übergängen und Datenfluss zwischen Frontend-Screens festgehalten. <i>Ergebnis:</i> Gemeinsames Verständnis für Frontend-Strukturierung mit IIRu5hII.
22.06.2026	D0rag3on	Ergebnis	Eintrag 59: JWT-Authentifizierung vollständig implementiert (PR #31): AuthController (Login, Logout, Register), AuthService, JwtUtil, JwtAuthFilter, AuthToken (Persistenz), AuthTokenRepository, SecurityConfig (stateless JWT), SecurityUserDetailsService. Außerdem: Foreign-Key-Beziehungen zwischen User, School und SchoolClass korrekt verknüpft.
22.06.2026	D0rag3on	Notiz	Eintrag 60: JWT-Secret aus application.properties als Umgebungsvariable externalisiert (PR #45). Der hartcodierte Wert wird jetzt aus \${JWT_SECRET} gelesen; kein sensibles Secret verbleibt im Repository. Entspricht dem definierten Sicherheitskonzept. Commit erscheint zweimal in der Historie (Cherry-Pick nach Merge-Verlust).

Datum	Autor	Klassifikation	Beschreibung / Ergebnis
2026-06-22	IIRu5hII	Recherche	Eintrag 61: Abgleich mit Backend-/Informationsmodell. Benötigte API-Daten: User/Profile, Kategorien, Szenarien, Fortschritt, Leaderboard, Attempt/Answers. <i>Ergebnis:</i> Vorbereitung auf Entfernung harter Mockdaten.
2026-06-22	IIRu5hII	Aufgaben	Eintrag 62: Mögliche DTO-/Repository-Grenzen identifiziert. UI soll langfristig über Repository/ViewModel-Daten arbeiten. <i>Ergebnis:</i> MVVM/Repository-Struktur als Zielarchitektur festgehalten.
2026-06-22	IIRu5hII	Offene Punkte	Eintrag 63: Mobile ViewModels und echte Repository-Implementierungen noch nicht vollständig vorhanden. <i>Ergebnis:</i> Für späteres Refactoring: UI-State-Klassen und Tests.
22.06.2026	OqIn	Versuchsaufbau	Eintrag 64: Kern-JPA-Entitäten implementiert (PR #44, f5af1c2). Refactoring der Stub-Entitäten von D0rag3on's Basis: User, School, SchoolClass, Scenario, Category, Attempt etc. mit Lombok-Annotations (@Data, @Builder, @Entity). Hibernate konfiguriert (spring.jpa.hibernate.ddl-auto=update). Erste Tests zum Laufen gebracht. <i>Notiz:</i> PR #44 wartete explizit auf PR #31 (D0rag3on's Auth): „Waiting for #31 to complete“.
22.06.2026	OqIn	Ergebnis	Eintrag 65: User-Management-API implementiert (PR #46, d8b7aa7). Neue Klassen: UserController (CRUD-Endpoints /api/v1/users), UserService/UserServiceImpl, UserRepository (Spring Data JPA), DTOs: CreateUserRequest, UpdateUserRequest, ResetPasswordRequest, UserResponse. Passwörter werden stets gehasht (BCrypt). <i>Ergebnis:</i> Vollständige CRUD-API für Benutzer; Lehrer können Passwörter zurücksetzen.
22.06.2026	OqIn	Ergebnis	Eintrag 66: Image- und ImageTag-Entitäten erstellt (PR #47, 2c9afa2). Image-Entität mit Metadaten (Beschreibung, Dateiname, Dateipfad), ImageTag-Entität (Verknüpfung Bild ↔ Nutzer ↔ Bewertung), TagValue-Enum (DANGEROUS/SAFE). <i>Ergebnis:</i> Datenmodell für kollaboratives Bild-Tagging im Backend.
22.06.2026	OqIn	Ergebnis	Eintrag 67: Image-Management-API implementiert (PR #48, 7a6cd2f). ImageController (Upload/Download/CRUD /api/v1/images), ImageService/ImageServiceImpl, ImageDataService/ImageDataServiceImpl (Datei-basierte Speicherung in data/images/), ImageRepository, ImageTagRepository. DTOs: CreateImageRequest, UpdateImageRequest, ImageResponse, ImageTagResultResponse. <i>Ergebnis:</i> Bilder können hochgeladen, abgerufen und verwaltet werden.

Datum	Autor	Klassifikation	Beschreibung / Ergebnis
22.06.2026	OqIn	Ergebnis	Eintrag 68: Bild-Tagging-API implementiert (PR #49, 92ab258). TagController (/api/v1/tags), TagService/ TagServiceImpl. Schüler können Bilder als DANGEROUS oder SAFE markieren; Duplikat-Tags werden mit DuplicateTagException abgelehnt. DTOs: SubmitTagRequest, TagResponse, ImageStatsResponse. <i>Ergebnis:</i> Kern-Spielmechanik (Bild bewerten) im Backend vollständig.
22.06.2026	OqIn	Versuchsaufbau	Eintrag 69: Nix Dev-Umgebung eingerichtet (PR #50, 3d72b56). devenv.nix: MariaDB, Java 25, Gradle deklarativ definiert; devenv.yaml und devenv.lock für Reproduzierbarkeit. <i>Ergebnis:</i> Einheitliche, reproduzierbare Entwicklungsumgebung für alle Teammitglieder ohne manuelle Tool-Installation.
22.06.2026	OqIn	Ergebnis	Eintrag 70: Schülerfortschritt- und Lehrer-Stats-API implementiert (PR #51, 5264857). Progress-Entität (Schüler ↔ Szenario: Score, Attempts, Completed), ProgressController/ProgressService/ProgressServiceImpl (/api/v1/progress). StatsController/StatsService/StatsServiceImpl (/api/v1/stats): Klassen-Statistiken pro Lehrer. DTOs: ProgressEntryResponse, ProgressSummaryResponse, ClassStatsResponse, StudentStatEntry. <i>Ergebnis:</i> Lehrer können Klassenergebnisse einsehen; Schüler-Fortschritt wird serverseitig persistiert.
22.06.2026	OqIn	Ergebnis	Eintrag 71: Build- & Release-Pipeline aufgebaut (PR #52, 95f8be5). Dockerfile für ssbackend, GitHub-Actions-Workflow azure-container-webapp.yml: Docker-Build → Push zu Azure Container Registry → Deploy zu Azure App Service. application-prod.properties mit Azure-SQL-Datasource-Konfiguration. <i>Ergebnis:</i> Automatisierter Deployment-Prozess vom Commit bis zur Produktionsumgebung auf Azure.
22.06.2026	OqIn	Problem	Eintrag 72: Azure-Container-Webapp-Workflow musste direkt nach Anlegen zweimal korrigiert werden (Commits 3ae7804, a9b36ee). Problem: Fehlende Umgebungsvariablen und inkorrekte Workflow-Trigger. <i>Ergebnis:</i> Workflow läuft stabil nach zwei Korrekturen.
2026-06-23	IIRu5hII	Ergebnis	Eintrag 73: Frontend-Prototyp vor Git-Historie konsolidiert. Wichtigste Screens mit Mockdaten verbunden. <i>Ergebnis:</i> Grundlage für anschließende Integration-Commits.
2026-06-23	IIRu5hII	Problem	Eintrag 74: Frühe Arbeiten ohne Commits — technische Historie nicht vollständig aus Git rekonstruierbar. <i>Ergebnis:</i> Laborbuch rekonstruiert frühe Schritte aus Quellcode und Mockups.

Datum	Autor	Klassifikation	Beschreibung / Ergebnis
2026-06-23	IIRu5hII	Offene Punkte	Eintrag 75: Backend-Anbindung, echte Auth, Bilddaten und persistenter Fortschritt müssen angebunden werden. <i>Ergebnis:</i> Nächster Fokus: API/Retrofit und Integration.
23.06.2026	OqIn	Problem	Eintrag 76: Spring-Boot-Tests liefen nicht (Commit-Kontext PR #53, 3cc8d2a). Ursachen: Fehlende H2-Testdatenbank-Konfiguration, Spring-Security-Testkonfiguration nicht gesetzt, Testfixtures unvollständig. <i>Lösung:</i> H2 In-Memory für Testprofil konfiguriert, @SpringBootTest-Annotations ergänzt, 17 Testklassen lauffähig.
23.06.2026	OqIn	Ergebnis	Eintrag 77: CI-Test-Workflow erstellt (PR #56, 63756d9). .github/workflows/test.yml: Gradle-Tests auf jedem Push und Pull-Request automatisch ausgeführt. <i>Ergebnis:</i> Kontinuierliche Qualitätssicherung; Regressions-Erkennung bei jedem Commit.
23.06.2026	OqIn	Ergebnis	Eintrag 78: Mobile-UI-Clickdummy vollständig implementiert (PR #54, f9ab2db). Ca. 4 072 Zeilen Kotlin/Compose neu hinzugefügt, 27 Dateien geändert. Screens: AuthScreen (Login/Register), HomeScreen (Level, Punkte, Streak, Kategorien), ScenariosScreen (Filter, Suche), ScenarioDetailScreen, ScenarioPlayScreen (Gameplay, Antwort-Buttons, Feedback), RankingScreen (Scope-Tabs, Top-3), ProfileScreen mit Unterseiten. Navigation via NavHost/BottomBar, Compose-Theme mit SafetySpot-Farben, Mock-Daten für vollständige Offline-Durchläufe. <i>Ergebnis:</i> Vollständig klickbarer Prototyp zum Demo-Termin bereit.
2026-06-24	IIRu5hII	Ergebnis	Eintrag 79: Commit 7d86b5d: Retrofit kann nun auch Plain Text lesen. Retrofit Scalars, DTOs, Auth/Login, Backend-Bilder sowie Home/Szenarien aus Backend vorbereitet und angebunden. <i>Ergebnis:</i> Beginn der konkreten Mobile-Backend-Integration.
2026-06-24	IIRu5hII	Aufgaben	Eintrag 80: DTOs und API-Kommunikation für Login/Auth und Datenabruf geprüft. <i>Ergebnis:</i> Vorbereitung auf Zusammenspiel ssmobile ↔ ssbackend.
2026-06-24	IIRu5hII	Ergebnis	Eintrag 81: Backend-Bilder als Quelle für Szenario-/Aufgabenbilder berücksichtigt. <i>Ergebnis:</i> Wichtig für spätere Hotspot-Gefahrensituationen.
2026-06-24	IIRu5hII	Ergebnis	Eintrag 82: Home- und Szenarienbereiche auf spätere Backend-Daten vorbereitet. <i>Ergebnis:</i> Mockdaten bleiben als Fallback/Demo nützlich.

Datum	Autor	Klassifikation	Beschreibung / Ergebnis
2026-06-24	IIRu5hII	Ergebnis	Eintrag 83: Commit 09beaca: Merge von origin/main in frontend-functionality. Übernommen: Test-Workflow, README, DemoDataService, AdminShellCommands. <i>Ergebnis:</i> Frontend-Branch mit aktuellem main synchronisiert.
2026-06-24	IIRu5hII	Problem	Eintrag 84: Merges können Konflikte zwischen Frontend-Prototyp und Backend-/DemoDataService-Änderungen verursachen. <i>Ergebnis:</i> Nach Merge geprüft, dass relevante Änderungen erhalten bleiben.
2026-06-24	IIRu5hII	Bewertung	Eintrag 85: CI/Test-Workflow und AdminShellCommands wichtig für Teamarbeit. <i>Ergebnis:</i> Projekt reproduzierbarer und wartbarer.
24.06.2026	OqIn	Ergebnis	Eintrag 86: Admin-Shell und Demo-Daten implementiert (PR #58, 8dc1fe1). AdminShellCommands (Spring Shell): interaktive CLI-Kommandos (create-school, create-class, create-user, seed, list-users etc.). DemoDataService: automatischer Seed beim App-Start (Schule, Klassen, Schüler, Lehrer, Bilder, Tags). <i>Ergebnis:</i> Backend kann ohne manuelle SQL-Statements mit realistischen Testdaten befüllt werden.
24.06.2026	OqIn	Notiz	Eintrag 87: Temporärer Workaround in ssmobile eingeführt (Commit 221ef31) und am 27.06.2026 wieder rückgängig gemacht (Commit 7a80d24). Hintergrund: Schnelle Umgehung eines Kompatibilitätsproblems zwischen Frontend-Functionality-Branch und Backend-Änderungen während aktiver Merge-Phase. <i>Ergebnis:</i> Nur temporär; saubere Lösung nach Merge eingespielt.
2026-06-25	IIRu5hII	Ergebnis	Eintrag 88: Commit 5a24e56: Merge branch main into frontend-functionality. DemoDataService-Änderungen übernommen. <i>Ergebnis:</i> Datenbasis/Seed-Daten mit Frontend-Branch kompatibel.
2026-06-25	IIRu5hII	Ergebnis	Eintrag 89: Commit d804323: weiterer Merge von main in frontend-functionality. Backend-/Security-Konfiguration übernommen. <i>Ergebnis:</i> Auth-relevante Konfiguration im Branch vorhanden.
2026-06-25	IIRu5hII	Aufgaben	Eintrag 90: Nach Merges geprüft, ob Frontend-Navigation, Demo-Daten und Backend-Konfiguration zusammenpassen. <i>Ergebnis:</i> Keine Regression im klickbaren Prototyp.
2026-06-25	IIRu5hII	Ergebnis	Eintrag 91: Commit 41832d1: Kategorie-Icons, Punkte-/Suchicon, Weitermachen-Logik, lokaler Fortschritt sowie Profilname/Avatar-Fallback ergänzt. <i>Ergebnis:</i> UI wirkt vollständiger und robuster.
2026-06-25	IIRu5hII	Lösung	Eintrag 92: Fallbacks für Profilname und Avatar eingeführt. <i>Ergebnis:</i> App bleibt bei fehlenden Backend-Daten nutzbar.

Datum	Autor	Klassifikation	Beschreibung / Ergebnis
2026-06-25	IIRu5hII	Ergebnis	Eintrag 93: Weitermachen-Logik verbessert: Home kann begonnenes Szenario anzeigen und Nutzer zurück in den Lernfluss führen. <i>Ergebnis:</i> Erhöht Nutzerbindung und passt zur Gamification-Idee.
2026-06-25	IIRu5hII	Test/Beobachtung	Eintrag 94: Nach Icon- und Fortschrittsänderungen wurden Home, Szenarien, Profil und Ranking manuell geprüft. <i>Ergebnis:</i> Keine offensichtlichen Brüche im Hauptfluss.
25.06.2026	OqIn	Ergebnis	Eintrag 95: Demo-Daten korrigiert (PR #59, e766cb6). Fehler im DemoDataService: falsche Beziehungs-IDs und inkonsistente Enum-Werte behoben. <i>Ergebnis:</i> Backend startet mit korrekten und konsistenten Testdaten.
25.06.2026	OqIn	Ergebnis	Eintrag 96: H2-Konsole für Dev-Profil aktiviert (PR #60, c24bcdcf). <code>spring.h2.console.enabled=true</code> in <code>application-dev.properties</code>. <i>Ergebnis:</i> Direkter Datenbankzugriff über Browser während Entwicklung erleichtert Debugging.
25.06.2026	OqIn	Ergebnis	Eintrag 97: Dateibasierte Bildspeicherung implementiert (PR #61, 659fa1a). <code>ImageDataServiceImpl</code> schreibt und liest Binärdaten unter <code>ssbackend/data/images/</code>. Verzeichnis mit <code>gitignore</code>-Ausnahme für Demo-Daten. <i>Ergebnis:</i> Bilder werden persistent auf dem Dateisystem gespeichert und per REST-Endpoint ausgeliefert.
25.06.2026	OqIn	Aufgaben	Eintrag 98: Debugging-Session Backend (Commit 1d41833). Untersuchung von Fehlern bei der Bild-Upload/Download-Kette zwischen <code>ssmobile</code> und <code>ssbackend</code>. <i>Ergebnis:</i> Probleme identifiziert und in Folgecommits behoben.
25.06.2026	OqIn	Ergebnis	Eintrag 99: Szenario-Demo-Bilder hinzugefügt (Commit 60f19a1). 10 PNG-Bilder (<code>scenario-00.png</code> bis <code>scenario-09.png</code>) unter <code>ssbackend/src/main/resources/demo/images/</code>. Diese werden vom <code>DemoDataService</code> beim Seed in die Datenbank geladen. <i>Ergebnis:</i> Vollständiger Demo-Betrieb mit realistischen Bild-Szenarien möglich; IIRu5hII kann Backend-Bilder direkt testen.
26.06.2026	D0rag3on	Ergebnis	Eintrag 100: School-/Klassen-Logik vollständig implementiert (PR #55): <code>SchoolClass</code>-Entität mit JPA-Beziehung zu <code>School</code>; <code>SchoolClassController</code>, <code>SchoolClassService</code>, <code>SchoolClassRepository</code>. Ergänzend: <code>SchoolController</code> und <code>SchoolService</code> fertiggestellt. DTOs: <code>CreateClass</code>, <code>CreateSchool</code>, <code>UpdateSchoolClass</code>, <code>UpdateSchoolRequest</code>.

Datum	Autor	Klassifikation	Beschreibung / Ergebnis
2026-06-26	IIRu5hII	Ergebnis	Eintrag 101: Commit edab15a: restliche Icons ergänzt. Profil-Icons, Filter-Icon und Avatar-Icon hinzugefügt; Profil- und Szenario-UI angepasst. <i>Ergebnis:</i> Letzter Feinschliff am visuellen Eindruck.
2026-06-26	IIRu5hII	Ergebnis	Eintrag 102: Profil-UI weiter an Mockups angepasst. Menüpunkte und Statuswerte visuell klarer erkennbar. <i>Ergebnis:</i> Profilseite für Schüler verständlicher und vollständiger.
2026-06-26	IIRu5hII	Ergebnis	Eintrag 103: Szenario-UI angepasst. Filter- und Szenariodarstellung optisch verbessert. <i>Ergebnis:</i> Szenarienliste leichter zu scannen; entspricht App-Stil.
2026-06-26	IIRu5hII	Bewertung	Eintrag 104: Die UI ist als funktionsfähiger Frontend-Prototyp vorzeigbar: Login/Gastzugang, Home, Szenarien, Spielscreen, Ranking und Profil. <i>Ergebnis:</i> Erfüllt MVP-Schwerpunkt aus Projektvision.
2026-06-26	IIRu5hII	Offene Punkte	Eintrag 105: Echte Persistenz und vollständige Backend-Anbindung müssen weiter stabilisiert werden. <i>Ergebnis:</i> Nächste Arbeit: Repository/ViewModel-Schicht, Auth-State, Tokenhandling, Cache.
2026-06-26	IIRu5hII	Offene Punkte	Eintrag 106: Lehrer-/Adminbereich im Mobile-Frontend noch nicht umgesetzt. <i>Ergebnis:</i> Für später: Klassen, Schülerverwaltung, Szenario-Editor, Statistiken.
2026-06-26	IIRu5hII	Offene Punkte	Eintrag 107: Offline-Modus konzeptionell vorgesehen, aber nicht umgesetzt. <i>Ergebnis:</i> Mit Room/DataStore/Pending Attempts planen.
2026-06-26	IIRu5hII	Schlussfolgerung	Eintrag 108: Mock-first-Ansatz hat sich bewährt: trotz fehlender früher Commits schnell nutzbarer Prototyp mit Design, Navigation und Lernflow. <i>Ergebnis:</i> Für finale Abgabe: Git-Commits künftig kleinschrittiger.
26.06.2026	Oqln	Ergebnis	Eintrag 109: Projekt-weite Code-Formatierung angewendet (PR #62, 3bda336). Gradle-Format-Task über alle Java-Dateien in ssbackend/ ausgeführt; einheitlicher Code-Stil durchgesetzt. <i>Ergebnis:</i> Diffs in Folge-PRs werden sauberer; Review-Aufwand sinkt.
27.06.2026	Oqln	Ergebnis	Eintrag 110: Szenario-Bilder aktualisiert (Commit 4736baf). Bilder-Set überarbeitet und mit dem aktuellen Stand des Backends abgeglichen. <i>Ergebnis:</i> Demo-Bilder im Frontend und Backend konsistent.

Datum	Autor	Klassifikation	Beschreibung / Ergebnis
27.06.2026	OqIn	Ergebnis	Eintrag 111: Android-APK-Release-Workflow erstellt (PR #64/#65, fbaf527, a54247d, 485ea21). android-apk-release.yml: automatischer APK-Build bei Git-Tag; APK als GitHub-Release-Asset veröffentlicht. Zwei Korrekturen nötig (PR #64 initiale Erstellung, PR #65 Fixup). <i>Ergebnis:</i> Vollständige Release-Pipeline für ssmobile; APK bei jedem Tag-Push automatisch verfügbar.
27.06.2026	OqIn	Notiz	Eintrag 112: Branch frontend-functionality nach umfangreichen Merges und Koordination mit IIRu5hll in main zusammengeführt (PR #63, 0027401). Der PR enthielt Commits beider Autoren (IIRu5hll's Icons/Weitermachen-Logik und OqIn's Debugging/Bilder/Workaround-Rücknahme). <i>Ergebnis:</i> Finaler Stand der mobilen Frontend-Funktionalität in main integriert; GitHub-Release v0.0.0 erstellt.

3 Zusammenfassung

D0rag3on: Backend-Grundgerüst, Maven-zu-Gradle-Migration, JWT-Authentifizierung, globales Error-Handling und School-/Klassen-Verwaltung.

IIRu5hll: Android-Frontend (Kotlin/Compose) – Design-System, Navigation, alle Screens (Auth, Home, Szenarien, Gameplay, Ranking, Profil) sowie Backend-Integration und UI-Feinschliff.

OqIn: Projektinfrastruktur, vollständige Backend-API (User, Image, Tag, Progress, Stats), CI/CD-Pipelines (Azure, Android), Mobile-Clickdummy und Demo-Daten.